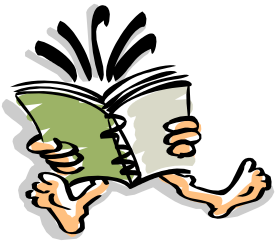
Analysis of Algorithms
CS 311

Recurrences



1) Write algorithm to Calculate the Factorial for a certain number

```
func factorial(n)
    if (n == 0)
        return 1
    return n * factorial (n- 1)
```

2) Write the recurrence relation.-

$$T(n) = T(n-1) + c$$

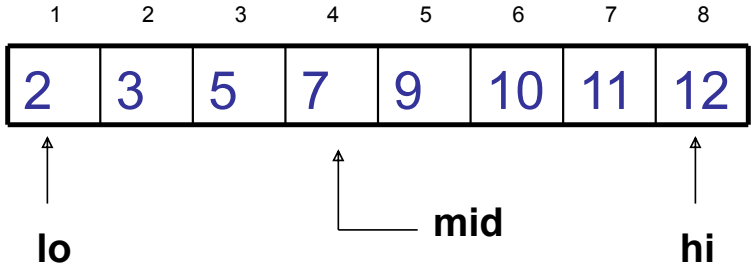
3) Calculate the time Complexity

Solve the recurrence.

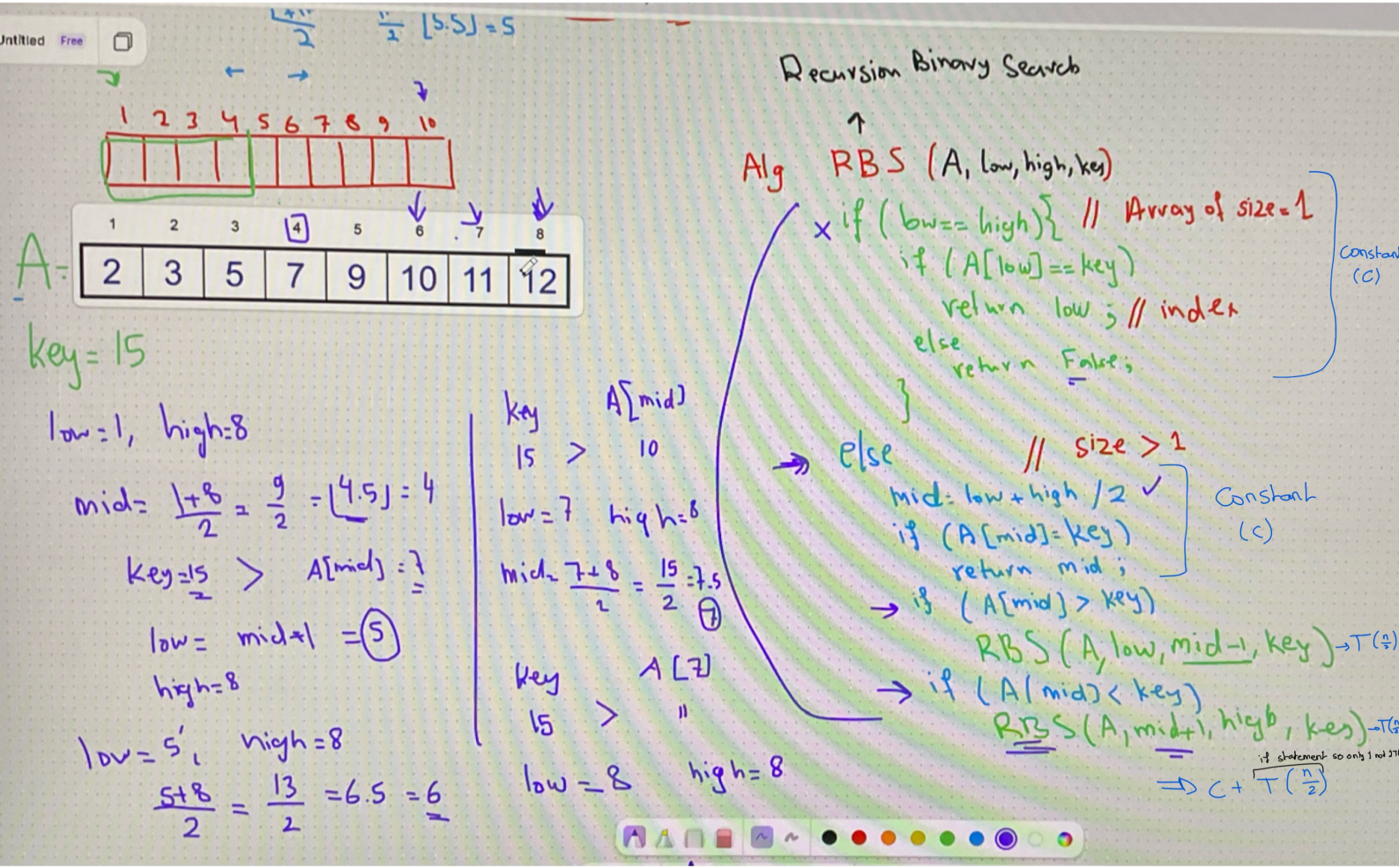
Recurrent Algorithms
BINARY-SEARCH

- for an ordered array A, finds if x is in the array A[lo…hi]

Alg.: BINARY-SEARCH (A, lo, hi, x)

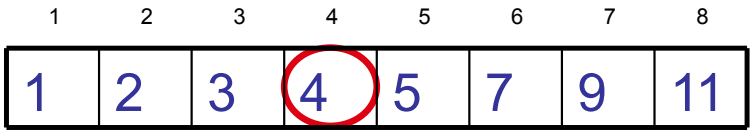



```
if (lo > hi)
    return FALSE
mid ← ⌊(lo+hi)/2⌋
if x = A[mid]
    return TRUE
if ( x < A[mid] )
    BINARY-SEARCH (A, lo, mid-1, x)
if ( x > A[mid] )
    BINARY-SEARCH (A, mid+1, hi, x)
```

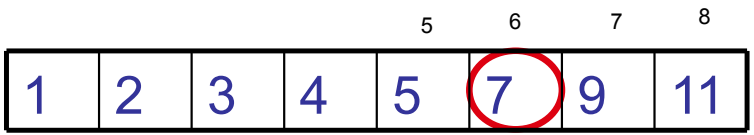

Untitled Free
Recursion Binary Search
Alg RBS (A, low, high, key)
if (low == high){ // Array of size = 1
if (A[low] == key)
return low ; // index
else
return False;
}
constant (C)
else // size > 1
mid = low + high / 2
if (A[mid] = key)
return mid ;
Constant (C)
if (A[mid] > key)
RBS (A, low, mid-1, key) → T(n/2)
if (A[mid] < key)
RBS (A, mid+1, high, key) → T(n/2)
⇒ C + T(n/2)
if statement so only 1 not 2 T
A =
1 2 3 4 5 6 7 8
2 3 5 7 9 10 11 12
key = 15
low = 1, high = 8
mid = 1+8/2 = 9/2 = ⌊4.5⌋ = 4
key = 15 > A[mid] = 7
low = mid+1 = 5
high = 8
low = 5, high = 8
5+8/2 = 13/2 = 6.5 = 6
key A[mid]
15 > 10
low = 7 high = 8
mid = 7+8/2 = 15/2 = 7.5 → 7
key A[7]
15 > 11
low = 8 high = 8

Example

- A[8] = {1, 2, 3, 4, 5, 7, 9, 11}
  - lo = 1 hi = 8 x = 7



mid = 4, lo = 5, hi = 8

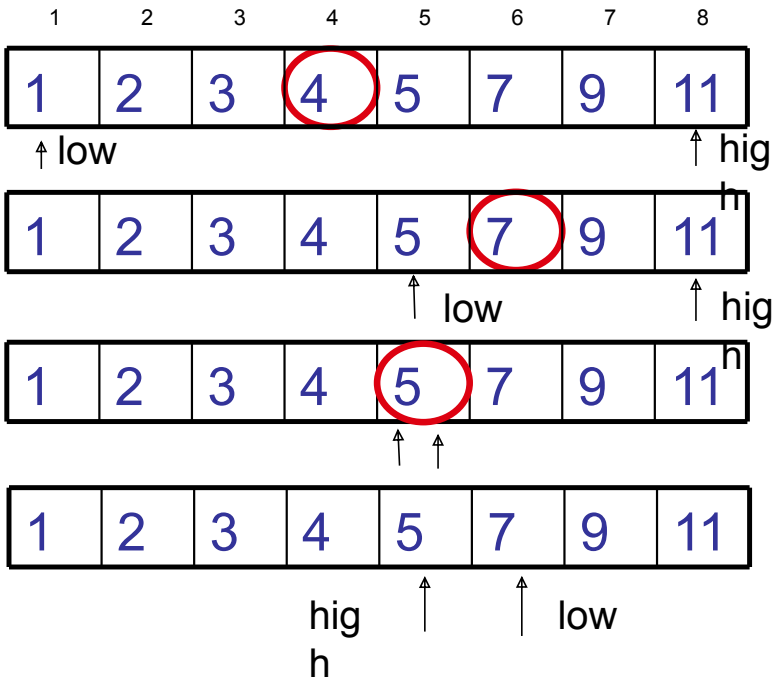


mid = 6, A[mid] = x
Found!

Another Example

- A[8] = {1, 2, 3, 4, 5, 7, 9, 11}

– lo = 1 hi = 8 x = 6



mid = 4, lo = 5, hi = 8

mid = 6, A[6] = 7, lo = 5, hi = 5

mid = 5, A[5] = 5, lo = 6, hi = 5
NOT FOUND!

Analysis of BINARY-SEARCH

Alg.: BINARY-SEARCH (A, lo, hi, x) ← A problem of size n

```
if (lo > hi)
    return FALSE
mid ← ⌊(lo+hi)/2⌋
if x = A[mid]
    return TRUE
if ( x < A[mid] )
    BINARY-SEARCH (A, lo, mid-1, x)
if ( x > A[mid] )
    BINARY-SEARCH (A, mid+1, hi, x)
```

← constant time: c1

← constant time: c2

← constant time: c3

← same problem of size n/2

← same problem of size n/2

$$T(n) = c + T(n/2)$$

★ $T(n) = T(n+1)$ goes to infinite but this could never happen

Also $T(2n)$ goes to infinite as well so neither will be in a question

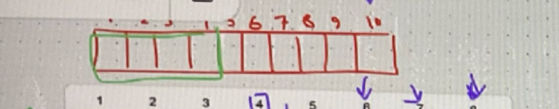
Usually it'll be subtraction or division

$T(0)=1$ $T(1)=1$

usually (depends on the code)

Untitled Free

Recursion Binary Search



key = 15, 8, 1, 10, 3

low = 1, high = 8

mid = $\frac{1+8}{2} = \frac{9}{2} = \lfloor 4.5 \rfloor = 4$

key = 15 > A[mid] = 7

low = mid+1 = 5

high = 8

low = 5, high = 8

$\frac{5+8}{2} = \frac{13}{2} = 6.5 = 6$

key	A[mid]
15	> 10

low = 7 high = 6

mid = $\frac{7+8}{2} = \frac{15}{2} = 7.5$

key	A[7]
15	> 11

low = 8 high = 8

Alg RBS (A, low, high, key)

```
if (low == high) {   // Array of size = 1
    if (A[low] == key)
        return low; // index
    else
        return False;
}                                   → C1
else                // size > 1
    mid = low + high / 2 ✓          → C2
    if (A[mid] = key)               → C3
        return mid;                 → C
    if (A[mid] > key)               →
        RBS(A, low, mid-1, key)     → T(n/2)
    if (A[mid] < key)               → C
        RBS(A, mid+1, high, key)    → T(n/2)
```

$\Rightarrow C + T(\frac{n}{2})$ $n>1$

1 $n=1$

$$T(n) = \begin{cases} C + T(\frac{n}{2}) & n>1 \\ 1 \quad T(1)=1 & n=1 \end{cases}$$

We'll use the iterative method:

$T(n) = C + T(\frac{n}{2}) \rightarrow T(\frac{n}{2}) = C + T(\frac{n}{2^2})$

$= C + C + T(\frac{n}{2^2}) \quad T(\frac{n}{2^2}) = C + T(\frac{n}{2^3})$

$= C + C + C + T(\frac{n}{2^3})$

$T(\frac{n}{2^k}) = C + T(\frac{n}{2^k})$

$\underbrace{C + \dots + C}_{k \text{ times}} + T(\frac{n}{2^k}) \Rightarrow \frac{n}{2^k} = 1$

$= kC + T(1)$

$= C \log n + 1$

$\log_2 n = \log_2 2^k$

$k = \log_2 n$

$$T(n) = \begin{cases} T(n-1) + 1 & n > 0 \\ 1 & n = 0 \end{cases}$$

$T(n) = T(n-1) + 1$

$T(n-1) = T(n-1-1) + 1$

$T(n-1) = T(n-2) + 1$

$T(n) = T(n-2) + 1 + 1$

$T(n-2) = T(n-2-1) + 1$

$T(n-2) = T(n-3) + 1$

$T(n) = T(n-3) + 1 + 1 + 1$

$\vdots$

k times

$T(n-k) + \underbrace{1 + 1 + \cdots + 1}_{k \text{ times}}$

$n - k = 0$

$n = k$

$T(0) + 1\,k$

$1 + n$

$T(n) = O(n)$

Try solving them as hw

1) $T(n) = \begin{cases} n + 2T\left(\frac{n}{2}\right) & n > 1 \\ 1 & n = 1 \end{cases}$

2) $T(n) = \begin{cases} T(n-1) + n & n > 0 \\ 0 & n = 0 \end{cases}$

3) $T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + \log n & n > 0 \end{cases}$

4) $T(n) = \begin{cases} 2T(n-1) + 1 & n > 0 \\ 1 & n = 0 \end{cases}$

$T(1) = 1$

1) $n + 2T\left(\frac{n}{2}\right)$

$T\left(\frac{n}{2}\right) = n + 2T\left(\frac{n}{2^2}\right)$

$T\left(\frac{n}{2^2}\right) = n + 2T\left(\frac{n}{2^3}\right)$

$n + n + 2^2 T\left(\frac{n}{2^2}\right)$

$n + n + n + 2^3 T\left(\frac{n}{2^3}\right)$

$kn + 2^k T\left(\frac{n}{2^k}\right)$

$\frac{n}{2^k} = 1$

$\log_2 n = \log_2 2^k$

$k = \log_2 n$

$kn + 2^{\log_2 n} T(1)$

$n \log n + n(1)$

$n \log n + n$

$\Rightarrow O(n) = n \log n$

$T(0) = 0$

2) $T(n-1) + n$

$T(n-1) = T(n-2) + n$

$T(n-2) = T(n-3) + n$

$T(n-2) + n + n$

$T(n-3) + n + n + n$

$\vdots$

$T(n-k) + kn$

$n - k = 0$

$n = k$

$T(0) + n \times n$

$0 + n^2$

$n^2 \Rightarrow O(n^2)$

$T(0) = 1$

3) $T(n-1) + \log n$

$T(n-1) = T(n-2) + \log n$

$T(n-2) + \log n + \log n - 1$

$T(n-2) = T(n-3) + \log n$

$T(n-3) + \log(n) + \log(n-1) + \log(n-2)$

$T(n-k) = \log 1 + \log 2 + \log 3 + \cdots + \log n$

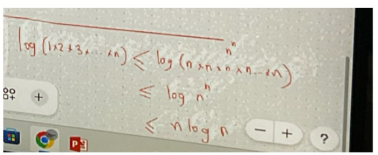
$n - k = 0$

$n = k$

$T(0) + \log 1 + \log 2 + \log 3 + \cdots + \log n$

$1 + \log(1 \times 2 \times 3 \cdots \times n)$

$1 + \log(n!)$



$T(0) = 1$

4) $2T(n-1) + 1$

$T(n-1) = 2T(n-2) + 2$

$= 2[2T(n-2) + 1] + 1$

$= 2^2 T(n-2) + 2 + 1$

$T(n-2) = 2T(n-3) + 3$

$= 2^2[2T(n-3) + 1] + 2 + 1$

$= 2^3 T(n-3) + 4 + 2 + 1$

$= 2^3 T(n-3) + 2^2 + 2^1 + 2^0$

$2^k T(n-k) + 2^{k-1} + \cdots + 2^{k-k}$

$n - k = 0$

$n = k$

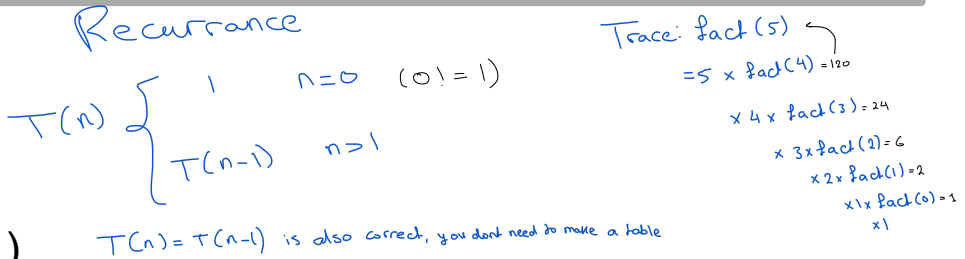
$2^n \underbrace{T(0)}_{1} + 2^{n-1} + \cdots + 2^{n-n}$

$= 2^n + 2^{n-1} + \cdots + 2^{n-n}$

$\Sigma 2^n \simeq 2^n$ so $O(n) = 2^n$

Analysis of factorial Algorithm

```
func factorial(n)
if (n == 0)
    return 1
return n * factorial (n- 1)
```



- Line 2 and 3 costs a constant time, c1 and c2.
- Line 4 calls factorial(n-1) which will take some time T(n-1). Also, the time it takes to multiply factorial(n-1) by n can be ignored once T(n-1) is used.
- Time for the whole function is just the sum: T(n) = c1 + c2 + T(n-1)
- This, in big-o notation, is reduced to T(n) = 1 + T(n-1)

Recurrences and Running Time

- Recurrences arise when an algorithm contains recursive calls to itself
- Recurrence: an equation that describes a function in terms of its value on smaller inputs.

have to be division or subtraction
multiplication or addition gives us huge number upto ∞ so its harder to solve them

$$T(n) = T(n-1) + n$$

- What is the actual running time of the algorithm?
  - Need to solve the recurrence

Example Recurrences

- $T(n) = T(n-1) + n$ $\Theta(n2)$
  - Recursive algorithm that loops through the input to eliminate one item
- $T(n) = T(n/2) + c$ $\Theta(\lg n)$
  - Recursive algorithm that halves the input in one step
- $T(n) = T(n/2) + n$ $\Theta(n)$
  - Recursive algorithm that halves the input but must examine every item in the input
- $T(n) = 2T(n/2) + 1$ $\Theta(n)$
  - Recursive algorithm that splits the input into 2 halves and does a constant amount of other work

Some of common recurrence and its solutions

☆ imp

label	recurrence	solution
A	`T(n) = T(n/2) + O(1)`	$O(\log n)$
B	`T(n) = T(n/2) + O(n)`	$O(n)$
C	`T(n) = 2T(n/2) + O(1)`	$O(n)$
D	`T(n) = 2T(n/2) + O(n)`	$O(n \log n)$
E	`T(n) = T(n-1) + O(1)`	$O(n)$
F	`T(n) = T(n-1) + O(n)`	$O(n^2)$
G	`T(n) = 2T(n-1) + O(1)`	$O(2^n)$

Methods for Solving Recurrences

- Iteration method
- Master method
- Substitution method

The Iteration Method

- Convert the recurrence into a summation and try to bound it using known series
  - Iterate the recurrence until the initial condition is reached.
  - Use back-substitution to express the recurrence in terms of *n* and the initial (boundary) condition.

Iteration Method – Example 1

$T(n) = c + T(n/2)$

$T(n) = c + T(n/2)$

$= c + c + T(n/4)$

$= c + c + c + T(n/8)$

$T(n/2) = c + T(n/4)$

$T(n/4) = c + T(n/8)$

Assume $n = 2k$

$T(n) = c + c + \ldots + c + T(1)$

(c repeated k times)

$= c\lg n + T(1)$

$= \Theta(\lg n)$

Iteration Method – Example 2

$T(n) = n + 2T(n/2)$

Assume: $n = 2k$

$T(n) = n + 2T(n/2)$

$T(n/2) = n/2 + 2T(n/4)$

$= n + 2(n/2 + 2T(n/4))$

$= n + n + 4T(n/4)$

$= n + n + 4(n/4 + 2T(n/8))$

$= n + n + n + 8T(n/8)$

$\ldots = in + 2iT(n/2i)$

$= kn + 2kT(1)$

$= nlgn + nT(1) = \Theta(nlgn)$

Master's method

- "Cookbook" for solving recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Idea: compare $f(n)$ with $n^{\log_b a}$

Master's method

- A utility method for for solving recurrences relations of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Case 1: if $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then: $T(n) = \Theta(n^{\log_b a})$

Case 2: if $f(n) = \Theta(n^{\log_b a})$, then: $T(n) = \Theta(n^{\log_b a} \lg n)$

Case 3: if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and if

$af(n/b) \leq cf(n)$ for some $c < 1$ and all sufficiently large n, then:

$T(n) = \Theta(f(n))$

regularity condition

Master's method

1. Check if the recurrences relation falls in Master's format
2. Extract a, b, and f(n) from the recurrences, where a ≥ 1, b > 1, and f(n) > 0
3. Determine nlogba
4. Compare f(n) with nlogba and determine the case

– Case 1: f(n) = O(nlogba -□) , upper bounded: f(n) < nlogba

– Case 2: f(n) = □(nlogba), tightly bound: f(n) = nlogba

1. Case 3: has two conditions

1. f(n) = □(nlogba +□), lower bounded: f(n) > nlogba

• af(n/b) ≤ cf(n), find a value for c < 1 makes the condition true

ive × Inbox (4,932) - majlan@p × Course: Design and Anal × Meet - fba-egbu-pgq × Untitl
ma.com/board/EC2fXTzsTmSy2gIq2VRrku/Untitled--Copy-?node-id=0-1&p=f&t=EzUIRyQWHwhkvtyB-0
Free

Could be given in the major (check with dr)

Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$a \geq 1$, $b > 1$ $\quad f(n) = n^k \log^p n$ $\quad k \geq 0$, p is real num

Case 1: $a > b^k$ Then $T(n) = \Theta(n^{\log_b a})$

Case 2: $a = b^k$

a) if $p > -1$, then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$

b) if $p = -1$, then $T(n) = \Theta(n^{\log_b a} \log \log n)$

c) if $p < -1$, then $T(n) = \Theta(n^{\log_b a})$

Case 3: $a < b^k$

a) if $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$

b) if $p < 0$, then $T(n) = O(n^k)$

Search

Example 1

$$T(n) = 2T(n/2) + \sqrt{n}$$

$a = 2, b = 2, \log_2 2 = 1$

Compare n with $f(n) = n^{1/2}$

$\square$ $f(n) = O(n^{1-\square})$ Case 1

$\square$ $T(n) = \square(n)$

Examples 2

$$T(n) = 9T(n/3) + n$$

Example 3

$$T(n) = 2T(n/2) + n$$

$a = 2, b = 2, \log_2 2 = 1$

Compare $n^{\log_2 2}$ with $f(n) = n$

$\Box$ $f(n) = \Theta(n)$ $\Box$ Case 2

$\Box$ $T(n) = \Theta(n \lg n)$

Example 4

$$T(n) = T(2n/3) + 1$$

Example 5

$$T(n) = 2T(n/2) + n2$$

$a = 2, b = 2, \log_2 2 = 1$

Compare n with $f(n) = n2$

$\Rightarrow f(n) = \Omega(n1+\varepsilon)$ Case 3 $\Rightarrow$ verify regularity cond.

$a\ f(n/b) \leq c\ f(n)$

$\Rightarrow 2\ n2/4 \leq c\ n2 \Rightarrow c = ½$ is a solution ($c<1$)

$\Rightarrow T(n) = \Theta(n2)$

Example 6

$$T(n) = 3T(n/4) + n \log n$$

The substitution method (not included in tests)

Mathematical Induction

1. Guess a solution
2. Use induction to prove that the solution works

Example 1

- T(n) = 2 T(n-1) +1
- T(0) = 0
- Prove by Induction, T(n) = $2n$ - 1

Example 2

- $T(n+1) = 3\,T(n) + 4$
- $T(1) = 1$
- Prove by Induction, $T(n) = 3n - 2$

The substitution method
Mathematical Induction

Guess a solution

- $T(n) = O(g(n))$
- Induction goal: apply the definition of the asymptotic notation
  - $T(n) \leq d\, g(n)$, for some $d > 0$ and $n \geq n0$
- Induction hypothesis: $T(k) \leq d\, g(k)$ for all $k < n$

(strong induction)

Prove the induction goal

- Use the **induction hypothesis** to find some values of the constants d and $n0$ for which the **induction goal** holds

Example: Binary Search

$$\mathbf{T(n) = c + T(n/2)}$$

Guess: $T(n) = O(\lg n)$

- Induction goal: $T(n) \leq d \lg n$, for some d and $n \geq n0$
- Induction hypothesis: $T(n/2) \leq d \lg(n/2)$

Proof of induction goal:

$$T(n) = T(n/2) + c \leq d \lg(n/2) + c$$

$$= d \lg n - d + c \leq d \lg n$$

if: $-d + c \leq 0, d \geq c$

Example

$$T(n) = 2T(n/2) + n$$

Guess: $T(n) = O(n \lg n)$

Induction goal: $T(n) \leq cn \lg n$, for some c and $n \geq n0$

Induction hypothesis: $T(n/2) \leq cn/2 \lg(n/2)$

Proof of induction goal:

$$T(n) = 2T(n/2) + n \leq 2c\,(n/2)\lg(n/2) + n$$

$$= cn \lg n - cn + n \leq cn \lg n$$

if: $- cn + n \leq 0 \Rightarrow c \geq 1$